



Lista de Exercícios 5

8 de abril de 2022

Engenharia de Computação

Lilian Giselly Pereira Santos

Pedro Henrique de Brito Nascimento

Projeto e Análise de Algoritmos - Rian Gabriel Pinheiro

Capítulo 5 - Programação Dinâmica

1. Inteiros positivos são arranjados em um triângulo equilátero com n números em sua base, como o mostrado na figura abaixo para $n = 4$. O problema é encontrar a menor soma em uma descida do ápice do triângulo até sua base por meio de uma sequência de números adjacentes (mostrados na figura pelos círculos). Projete um algoritmo de programação dinâmica para este problema.

Solução:

```
int min_tri_sum(vector<vector<int>>& triang) {
    int n = triang.size();
    vector<int> state(n, 0);
    vector<int> new_state(n+1, 0);
    state[0] = triang[0][0];
    for(int i = 1; i < n; i++){
        for(int j = 0; j < triang[i].size(); j++){
            if(j == 0){
                new_state[j] = triang[i][j] + state[j];
            }
            else if(j == triang[i].size() - 1){
                new_state[j] = triang[i][j] + state[j-1];
            }
            else{
                new_state[j] = triang[i][j] + min(state[j-1], state[j]);
            }
        }
        state = new_state;
    }
    int min_path = INT_MAX;
    for(int i = 0; i < n; i++){
        min_path = min(min_path, state[i]);
    }
    return min_path;
}
```

2. Algumas moedas são espalhadas nas células de um tabuleiro $n \times m$, uma moeda por célula. Um robô, localizado na célula superior esquerda do tabuleiro, precisa coletar o máximo de moedas possível e trazê-las para a célula inferior direita. Em cada etapa, o robô pode mover uma célula para a direita ou uma célula para baixo de sua localização atual. Quando o robô visita uma célula com uma moeda, ele pega a moeda. Elabore um algoritmo para encontrar o número máximo de moedas que o robô pode coletar e um caminho que ele precisa seguir para fazer isso.

Solução:

```
def robotPath(rows, cols, matrix):
    # matriz tem rows+1 linhas e cols+1 colunas
    robotPath = [[0]*(rows+1) for _ in range(cols+1)]

    for i in range(1, rows+1):
        for j in range(1, cols+1):
            # encontra o número máximo e adc 1 se contém uma moeda
            robotPath[i][j] = max(
                robotPath[i-1][j], robotPath[i][j-1])+matrix[i-1][j-1]

    print('As moedas coletadas pelo robô em cada nível são:\n', np.matrix(robotPath))
    print('Quantidade máxima de moedas coletadas:', robotPath[rows][cols])
    # imprime o caminho e o número máximo

# define uma matriz de exemplo
rows = 3
cols = 3
matrix = [[0]*rows for _ in range(cols)]
matrix = [[1, 1, 1],
          [1, 0, 1],
          [1, 1, 1]]
robotPath(rows, cols, matrix) # envoca a função do robô
```

3. Projete um algoritmo eficiente para encontrar o comprimento do caminho mais longo em um DAG. (Este problema é importante como um protótipo de muitos outros aplicativos de programação dinâmica, pois determina o tempo mínimo necessário para concluir um projeto que compreende tarefas de precedência restrita.)

Solução:

```

def addEdge(adj, u, v):

    adj[u].append(v)

def dfs(node, adj, arr, visited):

    visited[node] = True

    for i in range(0, len(adj[node])):

        if not visited[adj[node][i]]:
            dfs(adj[node][i], adj, arr, visited)

        arr[node] = max(arr[node], 1 + arr[adj[node][i]])

def lengthLongestPath(adj, k):

    arr = [0] * (k + 1)

    visited = [False] * (k + 1)

    for i in range(1, k + 1):
        if not visited[i]:
            dfs(i, adj, arr, visited)

    maxLength = 0

    for i in range(1, k + 1):
        maxLength = max(maxLength, arr[i])

    return maxLength

# Main Code

k = 5

adj = [[] for i in range(k + 1)]

# Exemplo
addEdge(adj, 2, 3)
addEdge(adj, 2, 4)
addEdge(adj, 4, 3)
addEdge(adj, 3, 5)
addEdge(adj, 4, 5)

print(lengthLongestPath(adj, k))

```

4. Uma subsequência contígua de uma lista S é uma subsequência feita de elementos consecutivos de S . Por exemplo, se S é

5, 15, -30, 10, -5, 40, 10,

então, 15, -30, 10 é uma subsequência contígua, mas 5, 15, 40 não é. Forneça um algoritmo de tempo linear para a seguinte tarefa:

Entrada: Uma lista de números $a_1, a_2, \dots, a_n$.

Saída: A subsequência contígua de soma máxima (a subsequência de tamanho zero tem soma zero).

Para o exemplo anterior, a resposta seria 10, -5, 40, 10, com uma soma de 55. (Dica: Para cada $j \in 1, 2, \dots, n$ considere subsequências contíguas terminando exatamente na posição j).

Solução:

```
def max_contiguous_sum(S):
    cur_max = -inf
    max_local = 0
    start_max = 0
    end_max = 0
    start = 0
    for i in len(S):
        max_local += S[i]

        if max_local > cur_max:
            cur_max = max_local
            start_max = start
            end_max = i

        if max_local < 0:
            max_local = 0
            start_max = i+1

    for i in range(start_max, end_max): print(S[i]+ " ")
```

5. Dadas duas strings $x = x_1x_2x_n$ e $y = y_1y_2y_m$, desejamos encontrar o comprimento da maior substring comum delas, isto é, o maior k para o qual existem índices i e j com $x_i x_{i+1} \dots x_{i+k-1} = y_j y_{j+1} \dots y_{j+k-1}$. Mostre como fazer isso em tempo $O(mn)$

Solução:

```

int longest_common_substr(char *str1, char *str2, int size1, int size2){
    int suff[size1+1][size2+1];
    int max_len=0;

    for(int i=0;i<size1;i++)
    {
        for(int j=0;j<size2;j++)
        {
            if(i==0||j==0) suff[i][j] = 0;
            else if(str1[i-1] == str2[j-1]){
                suff[i][j] = suff[i-1][j-1] + 1;
                max_len = max(max_len, suff[i][j]);
            }
            else
                suff[i][j]=0;
        }
    }
    return max_len;
}

```

6. Uma certa linguagem de processamento de strings oferece uma operação primitiva que divide uma string em dois pedaços. Como essa operação envolve copiar a string original, ela toma n unidades de tempo para uma string de tamanho n , não importa a posição do corte. Suponha, agora, que você queira quebrar a string em muitos pedaços. A ordem na qual os cortes são feitos pode afetar o tempo de execução total. Por exemplo, se você quiser cortar uma string de 20 caracteres nas posições 3 e 10, fazer o primeiro corte na posição 3 incorrerá em um custo total de $20+17 = 37$, enquanto fazer a posição 10 primeiro terá um custo melhor de $20 + 10 = 30$. Forneça um algoritmo de programação dinâmica que, dadas as posições de m cortes em uma string de comprimento n , encontre o custo mínimo de dividir a string nos $m + 1$ pedaços.

Solução:

```

#include <iostream>
#include <string.h>
#include <stdio.h>
#include <limits.h>
using namespace std;

int main()
{
    int cuts, length, k, split;
    while (scanf("%d%d", &length, &cuts) != EOF)
    {
        int arr[length + 1][length + 1];
        int cut[cuts];
        int i, j;

        memset(arr, 0, sizeof(arr));

        for (i = 0; i < cuts; i++)
            cin >> cut[i];

        for (split = 1; split <= length; split++)
        {
            for (i = 0, j = i + split; j <= length; j++, i++)
            {
                if (split == 1)
                    arr[i][j] = 0;
                else
                {
                    int min = INT_MAX;
                    for (k = 0; k < cuts; k++)
                    {
                        if (cut[k] < j and cut[k] > i)
                        {
                            int cost = (j - i) + arr[i][cut[k]] + arr[cut[k]][j];

                            if (cost < min)
                                min = cost;
                        }
                    }
                    if (min >= INT_MAX)
                        arr[i][j] = 0;
                    else
                        arr[i][j] = min;
                }
            }
        }
        cout << arr[0][length] << endl;
    }
    return 0;
}

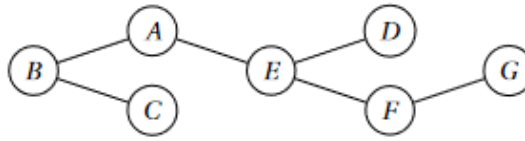
```

7. Uma cobertura de vértices de um grafo $G = (V, E)$ é um subconjunto de vértices $S \subseteq V$ que inclui ao menos uma extremidade de cada aresta de E . Forneça um algoritmo de tempo linear para a seguinte tarefa.

Entrada: Uma árvore não-direcionada $T = (V, E)$.

Saída: O tamanho da menor cobertura de vértices de T .

Por exemplo, na árvore abaixo, as possíveis coberturas de vértice incluem $\{A, B, C, D, E, F, G\}$ e $\{A, C, D, F\}$, mas não $\{C, E, F\}$. A menor cobertura de vértice tem tamanho 3: B, E, G



Solução:

```

class vertex-cover
{
    static int min(int x, int y){
        return (x < y) ? x : y;
    }

    class node{
        public int data;
        public int vertexcover;
        public node left, right;
    };

    static node createNode(int data){
        node el = new node();
        el.vertexcover = 0;
        el.left = el.right = null;
        el.data = data;
        return el;
    }

    static int vCover(node root){
        if (root == null)
            return 0;
        if (root.left == null &&
            root.right == null)
            return 0;

        if (root.vertexcover != 0)
            return root.vertexcover;

        int sizeWithRoot = 1 + vCover(root.left) +
                               vCover(root.right);

        int sizeNoRoot = 0;
        if (root.left != null)
            sizeNoRoot += 1 + vCover(root.left.left) +
                          vCover(root.left.right);
        if (root.right != null)
            sizeNoRoot += 1 + vCover(root.right.left) +
                          vCover(root.right.right);

        root.vertexcover = Math.Min(sizeWithRoot, sizeNoRoot);

        Console.WriteLine("O tamanho da menor cobertura de vértice é {0}", root.vertexcover);

        return root.vertexcover;
    }
}
  
```

8. Forneça um algoritmo de programação dinâmica (note que será pseudo-polinomial) para o problema SUBSET SUM.

Entrada: Um conjunto A com valores inteiros positivos, e um inteiro t.

Questão: Existe um subconjunto $A \subseteq A$ cuja soma dos valores seja exatamente t?

Solução:

```
bool isSubsetSum(int A[], int t, int k)
{
    bool subset[k + 1][t + 1];

    for (int i = 1; i <= t; i++)
        subset[0][i] = false;

    for (int i = 0; i <= k; i++)
        subset[i][0] = true;

    for (int i = 1; i <= k; i++)
    {
        for (int j = 1; j <= t; j++)
        {
            if (j < A[i - 1])
                subset[i][j] = subset[i - 1][j];
            if (j >= A[i - 1])
                subset[i][j] = subset[i - 1][j] || subset[i - 1][j - A[i - 1]];
        }
    }

    for (int i = 0; i <= k; i++)
    {
        for (int j = 0; j <= t; j++)
            printf("%4d", subset[i][j]);
        cout << "\n";
    }

    return subset[k][t];
}

int main()
{
    int A[] = {3, 34, 4, 12, 5, 2};
    int t = 9;
    int k = sizeof(A) / sizeof(A[0]);
    if (isSubsetSum(A, t, k) == true)
        cout << "Foi encontrado um subconjunto cuja soma dos valores é exatamente igual a 't'.";
    else
        cout << "Não há subconjunto com soma 't'.";
    return 0;
}
```

9. Dado um grafo simples $G(V, E)$ e dois vértices $s, t \in V$, projete um algoritmo de programação dinâmica que encontre a quantidade de caminhos simples distintos entre s e t

Solução:

```
vector<int> topo_sort(int freq[], int graph_size)
{
    queue<int> fila;

    for (int i = 0; i < n; i++){
        if (!freq[i]) fila.push(i);
    }

    vector<int> grafo_ordenado;

    while (!fila.empty()) {
        int u = fila.front();
        fila.pop();
        grafo_ordenado.push_back(u);

        for (int i = 0; i < grafo_inicial[u].size(); i++) {
            freq[grafo_inicial[u][i]]--;

            if (freq[grafo_inicial[u][i]]==0)
                fila.push(grafo_inicial[u][i]);
        }
    }
    return grafo_ordenado;
}

int caminhos_distintos(int destino, int n, int freq[])
{
    vector<int> grafo_ordenado = topo_sort(freq, n);

    int estados_anteriores[n] = { 0 };

    estados_anteriores[destino] = 1;
    for (int i = grafo_ordenado.size() - 1; i >= 0; i--) {
        pos = grafo_ordenado[i];
        for (int j = 0; j < grafo_inicial[pos].size(); j++) {
            estados_anteriores[pos] += estados_anteriores[grafo_inicial[pos][j]];
        }
    }
    return estados_anteriores[0];
}
```

10. Mostre um algoritmo de programação dinâmica para o problema do CAMINHO MÁXIMO entre dois vértices s e t (caminho simples). Qual a complexidade do algoritmo (tempo e memória)?

Solução: Um possível algoritmo para resolver tal tarefa é o algoritmo de Floyd-Warshall com uma pequena modificação (o algoritmo original foi feito para computar o caminho mínimo entre dois vértices), assim, o algoritmo ficaria:

```
def floyd_warshall(adj_dist_matrix):
    n = len(adj_dist_matrix[0])
    max_dist = adj_dist_matrix
    for k in range(1, n+1):
        for i in range(1, n+1):
            for j in range(1, n+1):
                max_dist[i][j] = max(max_dist[i][j], max_dist[i][k] + max_dist[k][j])
    return max_dist
```

Complexidade de Tempo: $O(n^3)$

Complexidade de Memória: $O(n^2)$

Outros algoritmos mais eficientes para fontes únicas, como o algoritmo de Bellman-Ford ou Dijkstra, também poderiam funcionar, a abordagem, neste caso, seria a mesma, maximizar o caminho ao vértice destino.